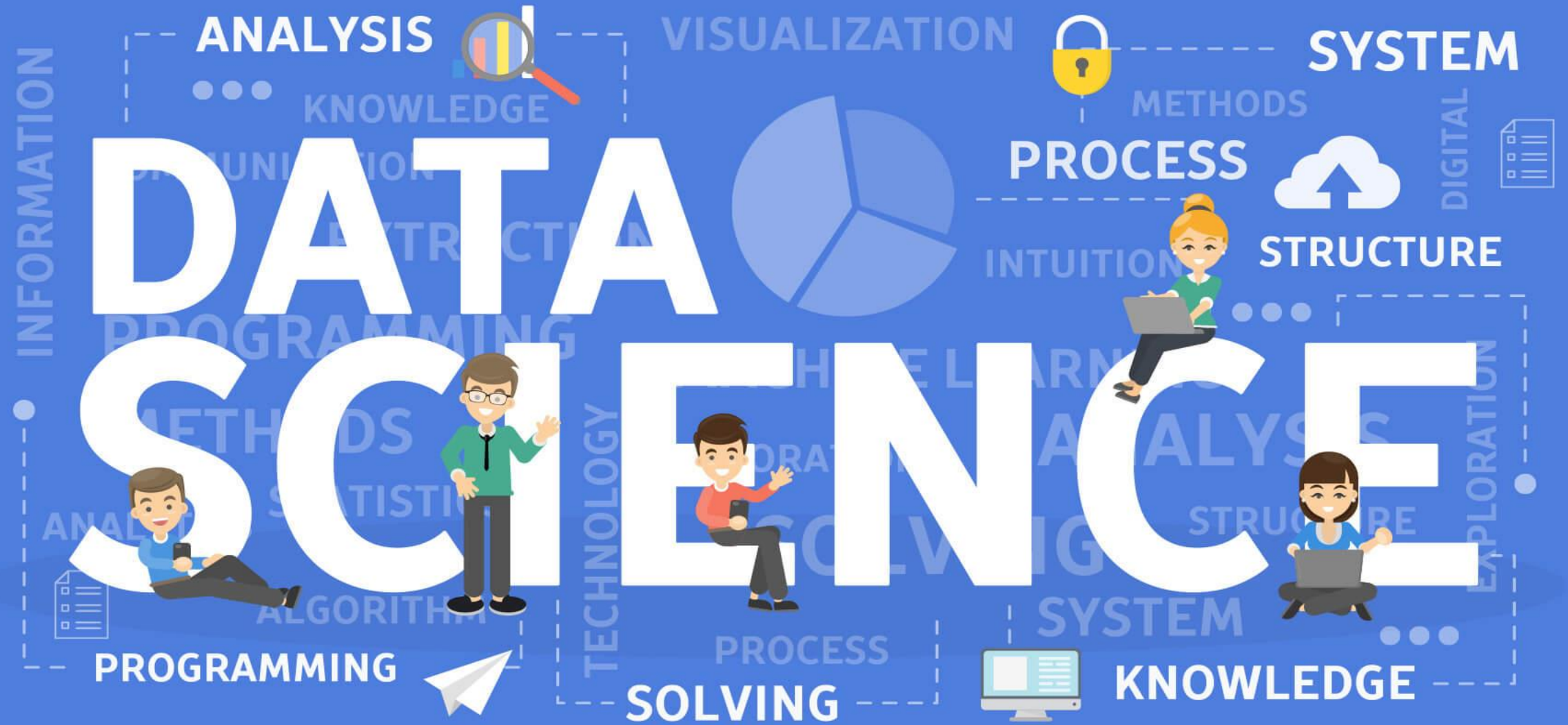




Ex:No.#03

Working with pandas Dataframe

PANDAS

Working with DataFrames

- ❖ A Pandas DataFrame is a two-dimensional table-like structure in Python where data is arranged in rows and columns.
- ❖ It's one of the most commonly used tools for handling data and makes it easy to organize, analyze and manipulate data.
- ❖ It can store different types of data such as numbers, text and dates across its columns.

The main parts of a DataFrame are:

Data: Actual values in the table.

Rows: Labels that identify each row.

Columns: Labels that define each data category.

Process

1

Creating DataFrames

Dicts, records, dtypes

2

Selecting & Filtering

loc, iloc, boolean masks, query

3

Missing Data

isna, fillna, dropna

4

GroupBy & Aggregation

agg, transform, multi-metric

5

Merging & Joining

inner, outer, left, concat

6

Reshaping

pivot, pivot_table, melt

7

Apply / Map / Strings

vectorized ops, custom functions

8

Time Series

resample, rolling, date parts

Common Data Frames Functions



Function	Purpose
head()	Display first 5 rows
tail()	Display last 5 rows
info()	DataFrame information
describe()	Statistical summary
shape	Number of rows and columns
columns	List of column names
loc[]	Access rows by label
iloc[]	Access rows by index
drop()	Remove rows/columns
rename()	Rename columns
sort_values()	Sort data
isnull()	Detect missing values
fillna()	Fill missing values
dropna()	Remove missing values
to_csv()	Save to CSV
read_csv()	Read CSV file

Creating DataFrames

The most common constructor: a dict of column names to lists.

pandas infers dtypes automatically — strings become

`Object`, numbers become `int64` / `float64`

, and

`pd.to_datetime()`

TIP

A list of dicts (one dict per row) mirrors JSON from an API response — often the shape you'll actually receive real-world data in.

```
df = pd.DataFrame({
    "name": ["Asha", "Bala", "Chitra"],
    "department": ["Sales", "Eng", "Sales"],
    "salary": [55000, 82000, 61000],
    "join_date": pd.to_datetime([
        "2019-03-15", "2017-07-01", "2015-11-20"
    ]),
})

# from a list of dicts (e.g. JSON)
pd.DataFrame([
    {"city": "Chennai", "temp_c": 34},
    {"city": "Mumbai", "temp_c": 31},
])
```

CHALLENGE 1 Build a small products table (product, price, stock) and filter to rows where `stock < 15`.

Selecting & Filtering

```
df.loc[0:2, ["name", "salary"]]
```

Label-based — rows & columns by name

```
df.iloc[1:3, 0:2]
```

Position-based — integer index

```
df[df.dept == "Eng"]
```

Boolean mask — single condition

```
df[(cond1) & (cond2)]
```

Combine masks with & / | (not and/or)

```
df.dept.isin(["HR", "Sales"])
```

Membership test across values

```
df.query("salary > 60000")
```

Readable string-expression filtering

CHALLENGE 2 Select employees in Engineering OR HR, aged over 30 — return just name and age.

Handling Missing Data

name	score	attempts
Asha	88	3
Bala	NaN	2
Chitra	76	NaN
Dev	NaN	4

Highlighted cells = missing values (NaN)

```
df.isna()           # boolean map
df[df.isna().any(axis=1)] # rows with nulls

# fill per-column
df.fillna({
    "score": df.score.mean(),
    "attempts": 0,
})

df.dropna()         # drop rows with any NaN
```

CHALLENGE 3 Fill missing score with 0, and missing attempts with the column median.

GroupBy & Aggregation

RAW ROWS

Sales	—	55k
Eng	—	82k
Sales	—	61k
Eng	—	95k
HR	—	48k
HR	—	52k

`groupby("department")`

GROUPS

Sales 55k, 61k
Eng 82k, 95k
HR 48k, 52k

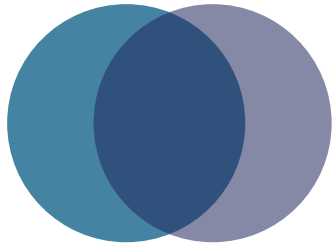
`.agg(mean, max, count)`

RESULT

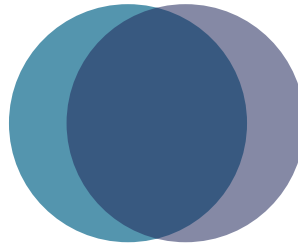
dept	avg	max	n
Sales	58k	61k	2
Eng	88.5k	95k	2
HR	50k	52k	2

CHALLENGE 4 Find the name of the highest-paid employee in each department (hint: `idxmax`).

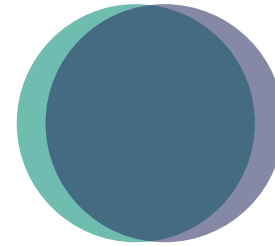
Merging & Joining

**INNER**

Only keys present in BOTH tables

**LEFT**

All rows from left, matched where possible

**OUTER**

All rows from either table

```
pd.merge(df_a, df_b, on="id", how="left")    # or "inner" / "outer" / "right"
```

```
pd.concat([df1, df2], ignore_index=True)    # stack rows on top of each other
```

CHALLENGE 5 Merge a `bonus_pct` table onto `employees` by department, and compute `bonus_amount`.

Reshaping: Long ↔ Wide

LONG FORMAT

month	region	rev
Jan	North	1000
Jan	South	1500
Feb	North	1100

`.pivot()``.melt()`

WIDE FORMAT

month	North	South
Jan	1000	1500
Feb	1100	1600

```
df.pivot(  
    index="month",  
    columns="region",  
    values="rev")  
  
df.pivot_table(  
    ..., aggfunc="sum")
```

pivot needs unique index/column pairs. **pivot_table** tolerates duplicates by aggregating them. **melt** reverses a pivot, turning columns back into rows.

CHALLENGE 6 Build a `pivot_table` of total revenue per region (rows) by month (columns), and add a Total column.

apply · map · string ops

Vectorized string ops

```
df.name.str.upper()  
df.name.str[0]
```

Fastest — operates on the whole column at once via C-level code.

.apply(fn, axis=1)

```
def band(row):  
    if row.salary >= 80000:  
        return "High"  
    ...  
df.apply(band, axis=1)
```

Row-wise custom logic — flexible but slower on large data.

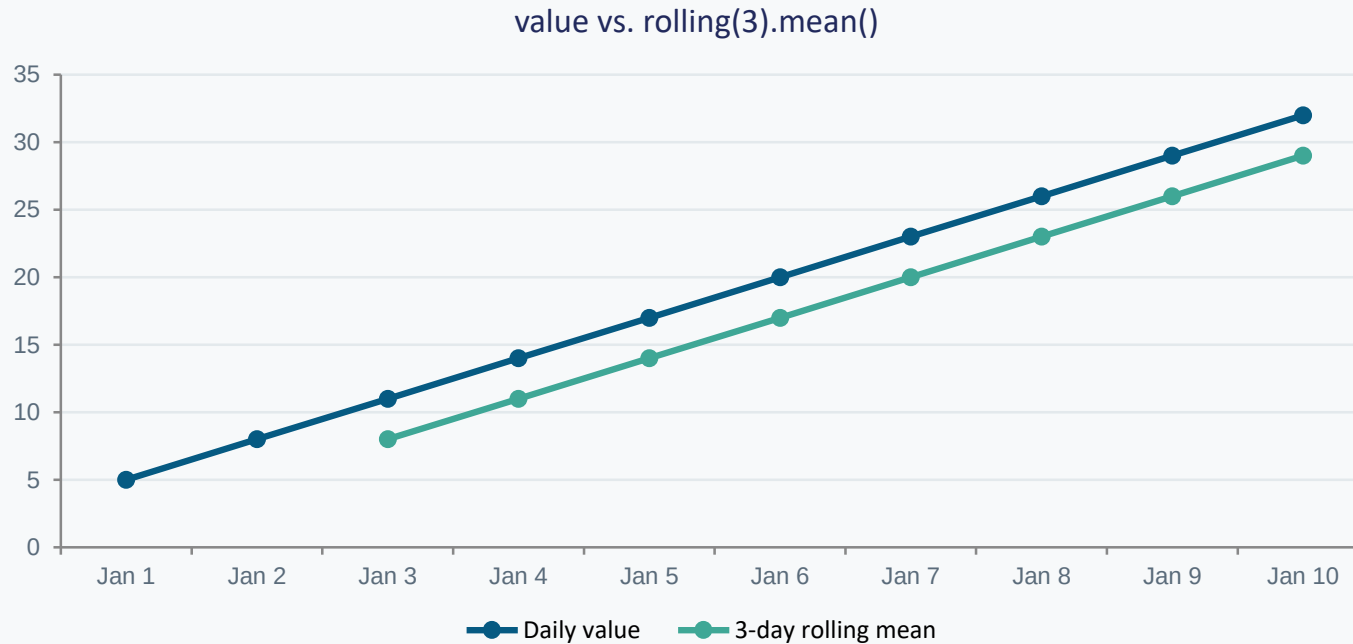
.map(dict)

```
df.dept.map({"Sales":"S",  
            "Engineering":"E"})
```

Simple value substitution on a single Series.

CHALLENGE 7 Add years_employed computed from join_date to today, rounded to 1 decimal.

Time Series Basics



```
ts = df.set_index("date")

# sum every 3-day bucket
ts.resample("3D").sum()

# smooth with a rolling window
ts.rolling(3).mean()

# pull date parts
ts.index.day_name()
```

CHALLENGE 8 Compute the cumulative sum of value over time, and find the date it first exceeds 50.

Create DataFrame using Dictionary

```
import pandas as pd
data={'name':['tom','nick','krish','jack'],'age':[20,21,19,18]}
df=pd.DataFrame(data)
print(df)
```

Select using column

```
import pandas as pd
data = {
    'name': ['jai', 'princy', 'gaurav', 'anuj'],
    'age': [27, 24, 22, 32],
    'address': ['delhi', 'kanpur', 'allahabad', 'kannauj'],
    'qualification': ['MA', 'MCA', 'Phd', 'BE']
}
df = pd.DataFrame(data)
print(df)
print(df[['name', 'qualification']])
```

Check Missing Values –isnull ()

```
import pandas as pd
import numpy as np
dic={'first':[100,90,np.nan,95],'second':[30,45,56,np.nan],
'third':[np.nan,40,80,98]}
df=pd.DataFrame(dic)
print(df.isnull())
```

Feature	Description
Pandas DataFrames	❖ Create a DataFrame with student details (Name, Age, Marks). ❖ Display the first 3 rows. ❖ Find students with marks greater than 80. ❖ Add a new column named Department. ❖ Rename Marks to Score. ❖ Sort students by Age in descending order. ❖ Remove one student record. ❖ Save the DataFrame to a CSV file. ❖ Read the CSV file back into a DataFrame. ❖ Display the statistical summary using describe().